

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design	25%	Design is robust,	Reasonable	Basic design	Weak design

Quality		feasible, and aligned to constraints	design	with gaps	
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

Student Name:	YADDALA KUSHAL KUMAR
Registration Number:	192524068
Course:	Cloud Computing and Big Data Analytics
Course Code:	CSA15
Assessment:	ASSESSMENT TOOL 1: Constraint-Based Problem Solving
Institution:	SIMATS ENGINEERING
Course Outcome:	CO5: Design big data processing systems using the Hadoop ecosystem including
Taxonomy:	Dave's Taxonomy: Articulation (Level 4)
Total Marks:	50

ASSESSMENT TASKS

Question 1

Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

Answer

HDFS Storage Design for High-Availability Media Platform

To meet the specific constraints of 99.9% availability, a maximum replication factor of 3, and cost-efficient scaling, the following HDFS-based architecture is proposed.

1. Identification and Prioritization of Constraints

- **Availability (>99.9%):** This is the primary constraint. In HDFS, this necessitates a **High Availability (HA) Cluster** configuration to eliminate the NameNode as a single point of failure (SPOF).
- **Replication Factor (Max 3):** This limits the brute-force approach to data durability. We must rely on rack awareness and efficient block placement rather than simply increasing copies.
- **Cost Efficiency:** As media data (video/images) grows exponentially, storage costs must be managed through **Heterogeneous Storage Management (HSM)** and **Erasure Coding (EC)** where applicable.

2. Proposed System Design

High Availability Architecture

To ensure >99.9% availability, the design utilizes **Quorum Journal Manager (QJM)** with two NameNodes (Active and Standby).

- **Zookeeper Failover Controller (ZKFC):** Monitors NameNode health and automates failover.
- **Journal Nodes:** A minimum of three Journal Nodes will synchronize edit logs between the Active and Standby NameNodes, ensuring zero data loss during transitions.

Rack-Aware Data Placement

With a strict replication factor of 3, the **Block Placement Policy** is configured as follows:

- **Replica 1:** Placed on the local node (or a random node in the local rack).
- **Replica 2:** Placed on a node in a different (remote) rack.
- **Replica 3:** Placed on a different node within the same remote rack.

This configuration protects against both individual node failure and entire rack failure, maximizing availability within the replication limit.

Storage Tiering and Cost Efficiency

To maintain cost efficiency during growth, the platform will implement **HDFS Storage Policies** :

- **HOT Tier (SSD/RAM):** For frequently accessed metadata and trending media content.
- **WARM Tier (Disk/HDD):** The default tier for standard media files using the 3x replication factor.
- **COLD Tier (Archive):** For older, rarely accessed media. Here, we transition from 3x replication to **Erasure Coding (EC)** . EC (e.g., Reed-Solomon 6,3) provides the same fault tolerance as 3x replication but reduces storage overhead from 200% to approximately 50%, significantly lowering the Total Cost of Ownership (TCO).

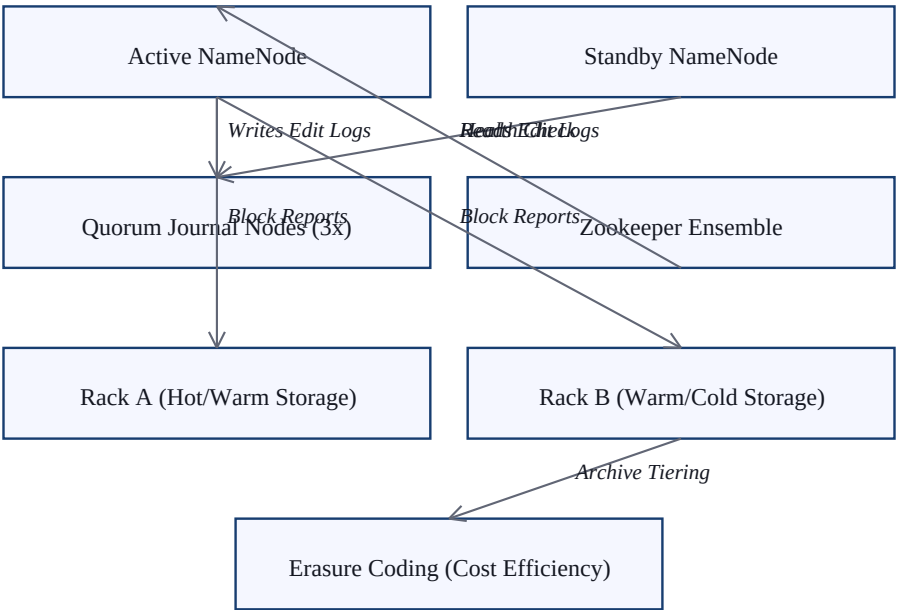
3. Application of Hadoop Concepts

- **HDFS Federation:** As the media platform scales, HDFS Federation can be used to add multiple independent NameNodes, allowing the namespace to scale horizontally without impacting the performance of the primary HA pair.
- **DataNode Balancer:** Regular execution of the HDFS Balancer ensures that storage growth is distributed evenly across the cluster, preventing hotspots and ensuring predictable performance.

4. Technical Justification

- **Availability:** The HA-QJM setup provides a sub-minute recovery time objective (RTO), which is essential for maintaining the 99.9% uptime SLA.
- **Constraint Adherence:** By using Erasure Coding for cold data, we respect the replication limit while achieving superior storage density.
- **Scalability:** The combination of Heterogeneous Storage and Federation ensures that as the media library grows, the system remains performant without requiring a linear increase in high-cost hardware.

Figure: High Availability HDFS Design for Media Platform



Question 2

A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

Answer

Job Design Proposal for Time-Constrained Log Processing

To process log data within a strict 20-minute batch window on a resource-constrained cluster, the design must prioritize **data locality**, **parallelism**, and **resource efficiency**. The following proposal outlines a MapReduce workflow optimized for these constraints.

1. Technical Constraints and Prioritization

- **Time Constraint (Primary):** The 20-minute window necessitates minimizing data movement and maximizing throughput.
- **Resource Constraint (Secondary):** Limited nodes require efficient memory management and avoiding "straggler" tasks that block the batch completion.
- **Data Volume:** Log data is typically high-volume and semi-structured, requiring efficient parsing.

2. Proposed Job Design Architecture

A. Data Ingestion and Storage (HDFS)

- **Block Size Optimization:** Set the HDFS block size to **256MB** (rather than 128MB) to reduce the total number of Map tasks, thereby decreasing the overhead of JVM startups on limited nodes.
- **Replication Factor:** Maintain a replication factor of 3 to ensure high data locality, allowing the

YARN Capacity Scheduler to assign tasks to nodes where data already resides.

B. Map Phase: Filtering and Pre-Aggregation

- **InputSplit Logic:** Use ``CombineFileInputFormat`` to merge small log files into larger splits. This prevents the cluster from being overwhelmed by too many small Map tasks.
- **In-Mapper Combining:** Implement a **Combiner class** (mini-reducer) to perform local aggregation of log keys (e.g., error codes or IP hits) before data hits the network. This significantly reduces the **Shuffle and Sort** overhead.

C. Shuffle and Sort Optimization

- **Compression:** Enable intermediate output compression using **Snappy** or **LZO** . These codecs offer a high compression-to-CPU ratio, reducing disk I/O and network congestion within the 20-minute window.
- **Memory Buffering:** Increase ``mapreduce.task.io.sort.mb`` to 200MB+ to minimize disk spills during the sort phase.

D. Reduce Phase: Final Aggregation

- **Reducer Scaling:** Set the number of reducers to **0.95 or $1.75 \times (\text{nodes} \times \text{cores})$** . This ensures all reducers launch immediately and finish in one wave, avoiding the latency of a second wave of tasks.

3. Task Distribution and YARN Resource Management

Task distribution is managed by **YARN (Yet Another Resource Negotiator)** to ensure the 20-minute SLA is met:

- **Resource Containers:** Each Map and Reduce task runs in a dedicated container. Given the limited nodes, ``yarn.scheduler.maximum-allocation-mb`` must be tuned to allow maximum concurrency without triggering swapping.
- **Speculative Execution:** Disable speculative execution if the cluster is at 100% utilization to prevent redundant tasks from stealing resources, or enable it strictly for the last 5% of tasks to mitigate stragglers.
- **Node Manager Locality:** YARN will prioritize **NODE_LOCAL** tasks, followed by **RACK_LOCAL** . By ensuring logs are distributed across the limited nodes, we minimize cross-switch traffic.

4. Justification of Design Choices

- **Why Combiners?** In log processing, many keys are repetitive. Reducing data volume at the source is the most effective way to meet time constraints on limited hardware.
- **Why Snappy Compression?** It is designed for speed rather than maximum compression ratio, which is critical for meeting a sub-20-minute batch window.
- **Why CombineFileInputFormat?** Log files are often generated in small increments. Processing them individually creates massive overhead; combining them ensures the limited nodes spend time processing data rather than managing task lifecycles.

Question 3

A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

Answer

Design of a YARN-Managed Scheduler Policy for Heterogeneous Workloads

To address the requirements of ETL, reporting, and machine learning (ML) jobs within a single cluster, the design utilizes a **Capacity Scheduler** architecture with hierarchical queues and preemption mechanisms. This approach ensures resource guarantees for critical tasks while maintaining high cluster utilization.

1. Identification of Technical Constraints

- **Fairness:** Ensuring that reporting and ML jobs are not starved by long-running ETL processes.
- **Priority for ETL:** ETL jobs are time-sensitive (SLA-driven) and must receive resources immediately upon submission.
- **Recovery from Node Failure:** The system must handle task re-execution without losing overall job progress, particularly for long-running ML models.
- **Resource Heterogeneity:** ML jobs typically require GPU/High-Memory resources, whereas ETL is I/O intensive.

2. Hierarchical Queue Design

A root-level partitioning strategy is implemented to isolate workloads while allowing for resource sharing:

- **ETL Queue (Priority: High, Capacity: 50%):**
 - Configured with a high `user-limit-factor` to allow a single production job to burst if needed.
- **Priority Scheduling:** Enabled within the queue to ensure critical data pipelines jump to the head of the pending list.
- **Reporting Queue (Priority: Medium, Capacity: 20%):**
 - Designed for low-latency, short-lived queries.
 - Implements `Fair Ordering` within the queue to prevent one large report from blocking others.
- **ML Queue (Priority: Low, Capacity: 30%):**
 - Targeted at long-running training jobs.
 - Utilizes **Node Labels** to steer ML tasks toward nodes equipped with specialized hardware (GPUs).

3. Application of YARN Concepts

Preemption and Resource Elasticity

To satisfy the **Priority for ETL** constraint, **Preemption** is enabled globally. If the ETL queue is below its guaranteed capacity and needs resources currently held by the ML queue, YARN will gracefully reclaim containers from ML tasks.

- **Graceful Decommissioning:** ML containers are given a timeout to checkpoint their state before being killed, supporting the **Recovery** constraint.

Scheduling Policies

- **Capacity Scheduler:** Chosen over the Fair Scheduler because it provides better predictability for production SLAs (ETL) through minimum guaranteed capacities.
- **Dominant Resource Fairness (DRF):** Configured as the resource calculator to account for CPU, Memory, and GPU requirements simultaneously, ensuring ML jobs don't monopolize one resource type while leaving others idle.

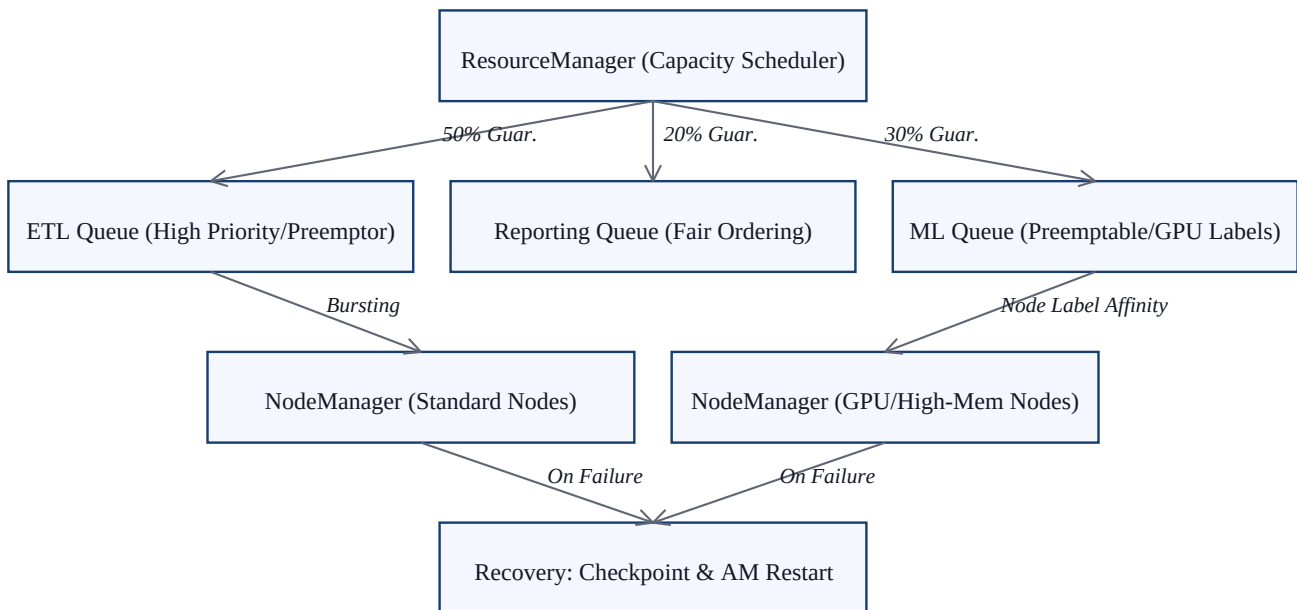
4. Fault Tolerance and Recovery Mechanisms

- **Application Master (AM) Restart:** The ``yarn.resourcemanager.am.max-attempts`` is increased for ETL jobs to ensure that if the AM fails during a node crash, the job automatically resumes.
- **Work-Preserving Restart:** Enabled to allow the ResourceManager to sync with NodeManagers upon recovery, ensuring that running containers (especially for ML) are not killed if the RM itself fails.
- **HDFS Checkpointing:** The ETL and ML logic must implement periodic state commits to HDFS. In the event of a node failure, YARN re-allocates the failed container, and the task resumes from the last successful checkpoint.

5. Technical Justification

- **Why Capacity over Fair?** Capacity scheduling allows for "hard" limits and guaranteed bandwidth for ETL, which is essential for business-critical data pipelines.
- **Why Preemption?** Without preemption, a large ML job could occupy the entire cluster for hours, causing ETL jobs to miss their SLAs regardless of their priority setting.
- **Why Node Labels?** ML workloads have specific hardware requirements that standard ETL does not; labels prevent resource wastage and performance degradation caused by running ML on underpowered nodes.

Figure: YARN Scheduler Architecture for Mixed Workloads



Question 4

An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

Answer

Proposed Constrained Architecture for Hybrid Data Ingestion

To address the ingestion of data from heterogeneous storage (SAN, NAS) and operational databases while adhering to constraints of **minimal duplication** and **backup reliability**, a tiered architecture leveraging the Hadoop ecosystem is proposed.

1. Data Source Integration Layer

- **Operational Databases (RDBMS):** Utilizes **Apache Sqoop** for structured data ingestion. Sqoop allows for incremental imports based on check-columns (timestamps or IDs), ensuring only new or modified records are ingested, directly addressing the **minimal duplication** constraint.
- **SAN/NAS (Unstructured/Semi-structured):** Utilizes **Apache Flume** or **NFS Gateway**. For SAN/NAS, mounting the storage via the HDFS NFS Gateway allows the big data platform to interact with existing storage as if it were a local file system, facilitating direct `distcp` (distributed copy) operations for bulk transfers.

2. Ingestion and Transport Layer (Apache Kafka)

- Acting as a buffer, **Apache Kafka** decouples the source systems from the storage layer. By implementing a pub-sub model, data can be streamed into the platform in real-time. Kafka's retention policies ensure that data is not lost during downstream downtime, providing an initial layer of **backup reliability**.

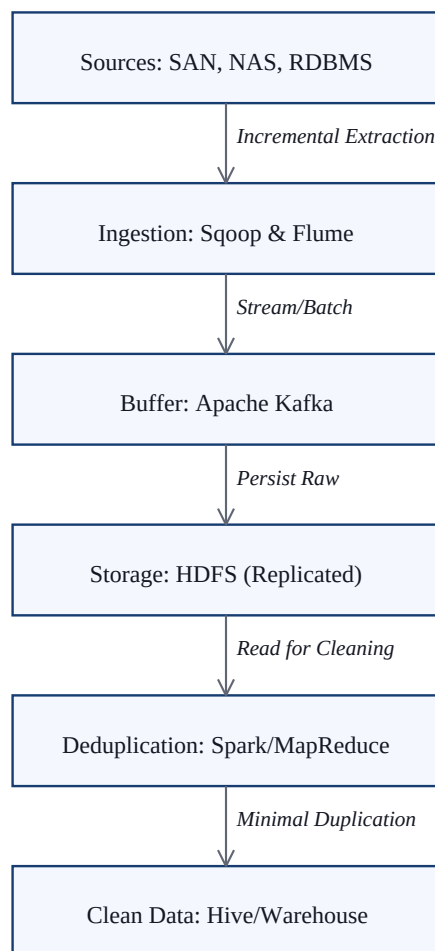
3. Storage and Management Layer (HDFS & Hive)

- **HDFS (Hadoop Distributed File System):** The primary storage layer. HDFS inherently provides reliability through a default **replication factor of 3**, ensuring that data is protected against hardware failure without requiring manual backup intervention.
- **Data Deduplication Logic:** To maintain minimal duplication, a **Medallion Architecture** (Bronze, Silver, Gold zones) is implemented. Raw data is landed in the Bronze zone; a MapReduce or Spark job then performs de-duplication and schema validation before moving it to the Silver zone.

4. Justification of Components

- **Apache Sqoop for Databases:** Chosen because it offloads the ETL overhead from the operational database to the Hadoop cluster. Its incremental load feature is the primary mechanism for meeting the "minimal duplication" constraint.
- **HDFS Replication for Backup:** Traditional backups of Petabyte-scale data are inefficient. HDFS's rack-aware replication provides high availability and disaster recovery natively, satisfying the "backup reliability" constraint without external third-party tools.
- **YARN (Yet Another Resource Negotiator):** Manages resources across the ingestion tasks. By setting resource quotas, we ensure that ingestion from SAN/NAS does not starve the operational database imports, maintaining system stability.
- **Data Checkpointing:** By using Kafka offsets and Sqoop last-value tracking, the architecture ensures "exactly-once" or "at-least-once" delivery semantics, preventing data gaps or redundant processing cycles.

Figure: Constrained Big Data Ingestion Architecture



Question 5

Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Answer

End-to-End Hadoop Ecosystem Design for Secure, High-Availability Data Pipelines

To address the requirements of secure data movement, high availability (HA), and seamless ETL integration, the proposed architecture leverages a multi-layered approach centered on the **Hadoop Distributed File System (HDFS)** and **Apache NiFi**.

1. Identification and Prioritization of Constraints

- **Secure Data Movement:** Ensuring data integrity and confidentiality during transit (TLS/SSL) and at rest (Encryption), alongside robust authentication (Kerberos).
- **High Availability (HA):** Eliminating Single Points of Failure (SPOF) for critical services like the NameNode and ResourceManager.
- **ETL Integration:** Utilizing Apache NiFi for low-latency, automated data ingestion and

transformation from diverse sources into the Hadoop ecosystem.

2. System Design and Component Integration

Data Ingestion Layer (Apache NiFi)

- **Role:** Acts as the primary ETL engine. NiFi processors are configured to pull data from RDBMS, IoT streams, or logs.
- **Security:** NiFi is integrated with **Apache Ranger** for fine-grained access control. Data provenance is maintained to track the lineage of every data packet.
- **Integration:** Uses the `PutHDFS` and `PutHiveQL` processors to move cleaned data into the storage layer.

Storage and Management Layer (HDFS & YARN)

- **High Availability:** The design employs **Quorum Journal Manager (QJM)** with two NameNodes (Active and Standby). Zookeeper Failover Controllers (ZKFC) monitor health and trigger automatic failover.
- **Security: HDFS Transparent Encryption** is implemented using a Key Management Server (KMS). Data is encrypted at the HDFS zone level, ensuring that even if physical disks are compromised, data remains unreadable.

Processing Layer (MapReduce & Spark on YARN)

- **Resource Management:** YARN is configured in HA mode with redundant ResourceManagers.
- **Authentication:** The entire cluster is secured via **Kerberos**, requiring valid tickets for any job submission or data access, preventing unauthorized "rogue" nodes from joining the cluster.

3. Satisfaction of Constraints

|Constraint|Implementation Strategy||:---|:---||**Secure Movement**|Use of Site-to-Site(S2S) protocol with TLS in NiFi; Kerberos authentication for all inter-service communication. ||**High Availability**|Active-Standby NameNode configuration using Zookeeper; YARN ResourceManager HA; NiFi Clustering. ||**ETL Integration**|NiFi provides a visual, flow-based interface to handle back-pressure and data prioritization before HDFS ingestion. ||**Data Governance**|**Apache Atlas** is integrated to provide metadata management and governance across the Hadoop stack. |

4. Technical Justification

- **Why NiFi?** Unlike traditional batch ETL, NiFi handles real-time data bursts and provides "Back Pressure" to prevent HDFS saturation during peak loads.
- **Why Kerberos?** It is the industry standard for distributed systems to verify identities in a non-secure network environment.
- **Why QJM for HA?** It avoids the need for expensive shared storage (like NFS) and provides a more robust synchronization of EditLogs between NameNodes, reducing the risk of data loss during failover.

Constraint Mapping and Implementation

Constraint	Technical Solution	Impact
Secure Data Movement	TLS/SSL + Kerberos + Ranger	End-to-end encryption and identity verification.
High Availability	Zookeeper + QJM + RM HA	99.99% uptime; no single point of failure.
ETL Integration	Apache NiFi S2S	Automated, scalable data ingestion with lineage.

Figure: Secure HA Hadoop Ecosystem with NiFi ETL

